SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0104**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I _M. Sanjay Krishna(192525421)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M Sanjay**

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	

10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

1. DOMAIN ANALYSIS

1.1 Problem Definition

Students may face academic difficulties because of low attendance, poor internal marks, incomplete assignments, learning difficulties, or poor previous academic performance. It can be difficult to manually analyze these factors and provide suitable academic advice. The proposed Student Academic Advisory Expert System uses a rule-based approach to analyze student information and recommend suitable academic interventions.

1.2 Objectives

• Identify students who require academic support.

• Analyze student academic conditions.

• Recommend suitable academic interventions.

• Represent knowledge using Prolog facts and production rules.

• Demonstrate forward chaining and backward chaining.

• Demonstrate unification and backtracking.

1.3 Input

Attendance, internal marks, assignment performance, learning difficulties, and previous academic performance.

1.4 Output

Recommendations such as improving attendance, extra academic practice, assignment support, remedial classes, faculty counselling, intensive academic support, regular monitoring, and study planning.

2. KNOWLEDGE ACQUISITION

The system represents academic knowledge using facts and IF–THEN production rules.

• Low attendance → Attendance improvement required.

• Low internal marks → Extra academic practice required.

• Incomplete assignments → Assignment support required.

• Learning difficulty → Remedial classes required.

• Poor previous performance → Faculty counselling required.

• Low attendance AND low internal marks → Intensive academic support required.

3. KNOWLEDGE REPRESENTATION

The knowledge is represented using Prolog facts and production rules.

Example IF–THEN rule:

IF attendance is low THEN student needs attendance improvement.

Prolog representation:

```
advice(Student, improve_attendance) :- attendance_low(Student).
```

4. COMPLETE SWI-PROLOG PROGRAM

```
% ========================================
% STUDENT ACADEMIC ADVISORY EXPERT SYSTEM
% ========================================


% ---------- FACTS ----------
attendance_low(student1).
internal_marks_low(student1).
assignment_incomplete(student1).
learning_difficulty(student1).
previous_performance_poor(student1).


attendance_good(student2).
internal_marks_good(student2).
assignment_complete(student2).
previous_performance_good(student2).


attendance_low(student3).
```

```
internal_marks_low(student3).
previous_performance_poor(student3).

% ---------- KNOWLEDGE BASE / RULES ----------
advice(Student, improve_attendance) :-
    attendance_low(Student).

advice(Student, extra_academic_practice) :-
    internal_marks_low(Student).

advice(Student, assignment_support) :-
    assignment_incomplete(Student).

advice(Student, regular_monitoring) :-
    attendance_low(Student),
    previous_performance_poor(Student).

advice(Student, study_plan) :-
    internal_marks_low(Student),
    previous_performance_poor(Student).

advice(Student, assignment_monitoring) :-
    assignment_incomplete(Student),
    internal_marks_low(Student).

advice(Student, continue_current_strategy) :-
    attendance_good(Student),
    internal_marks_good(Student),
    assignment_complete(Student),
    previous_performance_good(Student).
```

```
% ---------- BACKWARD CHAINING ----------
backward_chaining(Student, Advice) :-
    advice(Student, Advice).

% ---------- FORWARD CHAINING ----------
forward_rule(Student, improve_attendance) :-
    attendance_low(Student).

forward_rule(Student, extra_academic_practice) :-
    internal_marks_low(Student).

    write(':'), nl,
    forall(
        advice(Student, Advice),
        (
            write('- '),
            write(Advice),
            nl
        )
    ).

% ---------- EXPLANATION ----------
explain(Student, Advice) :-
    advice(Student, Advice),
    write('Reasoning: '),
    write(Student),
    write(' satisfies the conditions for '),
    write(Advice),
    write('.'), nl.
```

5. HOW TO RUN THE PROGRAM

?- [student_advisor].

Expected output:

true.

6. BACKWARD CHAINING DEMONSTRATION

Backward chaining starts with a goal and works backwards to check whether the required conditions are true.

?- backward_chaining(student1, improve_attendance).

true.

Reasoning: The goal improve_attendance requires attendance_low(Student). The fact attendance_low(student1) is true, so the conclusion succeeds.

7. FORWARD CHAINING DEMONSTRATION

Forward chaining starts with available facts and applies applicable rules to derive conclusions.

?- forward_chaining(student1, Advice).

Advice = improve_attendance ;

Advice = extra_academic_practice ;

Advice = assignment_support ;

Advice = remedial_classes ;

Advice = faculty_counselling ;

Advice = intensive_academic_support ;

Advice = regular_monitoring ;

Advice = study_plan ;

Advice = assignment_monitoring.

8. SYSTEM DEMONSTRATION

?- show_advice(student1).

Academic Advice for student1:

- improve_attendance

- extra_academic_practice

- assignment_support

- remedial_classes

- faculty_counselling

- intensive_academic_support

- regular_monitoring

- study_plan

- assignment_monitoring

9. UNIFICATION

Unification is the process by which Prolog matches terms and assigns values to variables.

?- advice(Student, improve_attendance).

Student = student1 ;

Student = student3.

Here Prolog unifies the variable Student with matching student facts.

10. BACKTRACKING

Prolog automatically searches for alternative solutions when the first solution is found.

?- advice(student1, Advice).

Advice = improve_attendance ;

Advice = extra_academic_practice ;

Advice = assignment_support ;

Advice = remedial_classes ;

...

The semicolon (;) requests another possible solution, demonstrating backtracking.

11. PROCEDURAL VS NON-PROCEDURAL PARADIGM

Procedural	Non-Procedural / Declarative
Specifies how to solve the problem.	Specifies what is true.
Uses sequence of instructions.	Uses facts and rules.
Programmer controls execution.	Inference engine performs reasoning.
Focuses on algorithms.	Focuses on knowledge and relationships.
Example: C/Python style.	Prolog is primarily declarative.

In this project, Prolog represents relationships between conditions and conclusions rather than explicitly specifying every execution step.

12. TEST CASES

Test Case	Student	Conditions	Expected Result
TC1	student1	Low attendance, low marks	Intensive academic support

TC2	student1	Incomplete assignments	Assignment support
TC3	student1	Learning difficulty	Remedial classes
TC4	student3	Low attendance	Improve attendance
TC5	student3	Poor previous performance	Faculty counselling
TC6	student2	Good academic performance	Continue current strategy

13. SAMPLE TEST QUERIES AND OUTPUTS

?- backward_chaining(student1, intensive_academic_support).

true.

?- backward_chaining(student1, remedial_classes).

true.

?- backward_chaining(student2, continue_current_strategy).

true.

?- backward_chaining(student2, remedial_classes).

false.

14. EVALUATION

• Correctness: Recommendations are generated according to predefined production rules.

• Coverage: The system covers attendance, internal marks, assignments, learning difficulties, and previous academic performance.

• Consistency: The same conditions produce the same conclusions.

• Reasoning Accuracy: Forward and backward reasoning can reach expected conclusions.

15. ADVANTAGES

• Easy to understand and demonstrate.

• Uses logical reasoning.

• Provides consistent recommendations.

• Knowledge can be expanded by adding rules.

• Demonstrates Prolog inference capabilities.

• Supports forward and backward reasoning.

16. LIMITATIONS

• The system depends on predefined knowledge.

• It cannot replace a qualified academic advisor.

• It does not learn automatically from new student data.

• Complex academic situations may require additional rules.

17. FUTURE ENHANCEMENTS

• Add GPA and subject-wise performance analysis.

• Include attendance percentage.

• Add a web-based user interface.

• Expand the knowledge base.

• Add automatic report generation.

• Integrate machine-learning techniques.

18. CONCLUSION

The Student Academic Advisory Expert System demonstrates how artificial intelligence can be used to provide academic recommendations using a rule-based approach. The system represents academic knowledge using Prolog facts and production rules and demonstrates forward chaining, backward chaining, unification, and backtracking. It also illustrates the difference between procedural and non-procedural approaches. The project provides a simple and effective demonstration of a rule-based expert system using SWI-Prolog.

19. GITHUB REPOSITORY STRUCTURE

```
Student-Academic-Advisory-Expert-System/
├── student_advisor.pl
├── README.md
├── report/
│   └── Expert_System_Report.pdf
├── screenshots/
│   ├── forward_chaining.png
│   ├── backward_chaining.png
│   └── test_cases.png
└── test_cases.txt
```

20. REFERENCES

1. SWI-Prolog documentation and learning resources.

2. Course assessment document: CO5 AT1 – Expert System Development Project.

3. Course concepts on production rules, forward chaining, backward chaining, unification, and backtracking.